ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

Ma Trận - Cơ sở của Tính Toán Khoa Học

Báo Cáo Đồ Án Số 1
Toán Ứng Dụng và Thống Kê - MTH00051

Sinh viên thực hiện:

24120010 - Trần Văn Tấn Khôi

24120028 - Cao Việt Cường

24120035 - Lê Thành Đạt

24120038 - Nguyễn Phú Đạt

24120054 - Mai Phan Nhật Hoàng

Giảng viên hướng dẫn:

Võ Nam Thục Đoan

Lê Nhựt Nam

Thành phố Hồ Chí Minh, Ngày 30 tháng 4 năm 2026

BẢNG PHÂN CHIA CÔNG VIỆC

MSSV	Họ và tên	Phân công công việc
24120010	Trần Văn Tấn Khôi	• Soạn kịch bản, ghi âm và xuất video Manim. • Viết mã nguồn cho các hoạt ảnh Manim. • Thiết lập repo, branch, mẫu PR và các file cần thiết khác
24120028	Cao Việt Cường	• Xây dựng các hàm cơ bản và thuật toán giải hệ phương trình trong phần 1. • Cài đặt bộ lặp Gauss-Seidel và đo hiệu năng. • Phân tích dữ liệu và độ ổn định ở phần 3.
24120035	Lê Thành Đạt	• Tổng hợp báo cáo cho phần 1, 2 và kết luận. • Cài đặt thuật toán phân rã SVD ở phần 2. • Viết mã nguồn cơ sở cho các hoạt ảnh Manim.
24120038	Nguyễn Phú Đạt	• Cài đặt phần định thức, tính ma trận nghịch đảo ở phần 1. • Viết notebook demo các hàm ở phần 1.
24120054	Mai Phan Nhật Hoàng	• Cài đặt thuật toán chéo hóa ma trận ở phần 2. • Tổng hợp phần chéo hóa cho báo cáo.

Link github đồ án: https://github.com/TanKhoiTV/matdecomp

Link video Manim: https://www.youtube.com/watch?v=AWdapXiePy8

MỤC LỤC

MỤC LỤC ii

DANH MỤC CÁC BẢNG v

DANH MỤC CÁC HÌNH ẢNH vi

1 Phép Khử Gauss và Các Ứng Dụng 1

1.1 Cơ sở lý thuyết 2

1.1.1 Dạng REF và RREF 2

1.1.2 Khử Gauss có chọn phần tử chốt 2

1.1.3 Tính định thức bằng khử Gauss 3

1.1.4 Ma trận nghịch đảo (Gauss-Jordan) 3

1.1.5 Hạng ma trận và các không gian con 3

1.2 Kết quả thực nghiệm và so sánh với Numpy 4

1.2.1 Mục tiêu 4

1.2.2 Các trường hợp kiểm thử 4

1.2.3 Đánh giá sai số 5

2 Phân Rã Ma Trận Và Trực Quan Hóa Với Manim 7

2.1 Phân rã giá trị suy biến (SVD) 8

2.1.1 Định nghĩa ma trận SVD 8

2.1.2 Tính chất của các giá trị suy biến 8

2.1.3 Mối liên hệ với giá trị riêng (Eigenvalues) 8

2.2 Ý nghĩa của phân rã SVD 9

2.2.1 Cơ chế hoạt động 9
2.2.2 Vì sao cơ chế này quan trọng? 10
2.3 Thuật toán giá trị riêng Jacobi 10
2.3.1 Định lý bất khả Abel 10
2.3.2 Chi tiết thuật toán 11
2.4 Chéo hóa ma trận 13
2.4.1 Cơ sở lý thuyết 13
2.4.2 Trường hợp đặc biệt: Ma trận đối xứng thực 13
2.5 Phương pháp tìm trị riêng và vector riêng 14
2.5.1 Thuật toán QR với kỹ thuật Deflation 14
2.5.2 Dịch chuyển Wilkinson (Wilkinson Shift) 15
2.5.3 Phân rã QR bằng Gram-Schmidt cải tiến 16
2.6 Kết quả 18
2.7 Trực quan hóa với Manim 19
2.7.1 Nội dung của video 19
2.7.2 Động lực thực hiện 19
2.7.3 Quy trình xây dựng video 20
2.7.4 Phân tích kỹ thuật chuyên sâu 21

3 Giải Hệ Phương Trình Và Phân Tích Hiệu Năng 23
3.1 Cơ sở lý thuyết 24
3.2 Phân tích hiệu năng 25
3.3 Phân tích độ ổn định 26
3.3.1 Hệ ma trận SPD (well-conditioned) 26
3.3.2 Hệ ma trận Hillbert (ill-conditioned) 27
3.4 Kết luận 28

4 Kết Luận 29

4.1 Tóm tắt kết quả đạt được 30
4.2 Khó khăn gặp phải 30
4.3 Bài học rút ra 31

DANH MỤC CÁC BẢNG

Bảng 1.1 Kết quả kiểm thử phương pháp Gauss 5
Bảng 1.2 Kết quả kiểm thử tính định thức 5
Bảng 1.3 Phân tích ảnh hưởng của hệ số điều kiện 6
Bảng 1.4 Kết quả kiểm thử hạng ma trận và không gian nghiệm 6
Bảng 2.1 Kết quả phân rã SVD cho các ma trận kiểm thử 18

DANH MỤC CÁC HÌNH ẢNH

Hình 2.1 Các bước phân rã SVD 9

Hình 3.1 Thời gian thực thi 25

Hình 3.2 Sai số trên hệ SPD 26

Hình 3.3 Sai số trên hệ Hillbert 27

CHƯƠNG 1

Phép Khử Gauss và Các Ứng Dụng

1.1. Cơ sở lý thuyết

1.1.1. Dạng REF và RREF

Cho ma trận $A \in \mathbb{R}^{m \times n}$. Thông qua các phép biến đổi dòng sơ cấp, ta có thể đưa ma trận về dạng bậc thang dòng (REF) hoặc dạng bậc thang dòng rút gọn (RREF).

- **REF (Row Echelon Form)**:
  - ❏ Các dòng toàn 0 nằm ở dưới.
  - ❏ Phần tử dẫn đầu (pivot) của mỗi dòng nằm của bên phải pivot của dòng trên.

$$\begin{bmatrix} a & b & c \\ 0 & d & e \\ 0 & 0 & f \end{bmatrix}$$

- **RREF (Reduced Row Echelon Form)**:
  - ❏ Thỏa mãn dạng ma trận dạng bậc thang dòng.
  - ❏ Mỗi phần tử pivot đều bằng 1.
  - ❏ Các phần tử khác trong cột chứa pivot đều bằng 0.

$$\begin{bmatrix} 1 & 0 & 2 & 0 \\ 0 & 1 & 3 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

REF thường được dùng trong phép khử Gauss, trong khi RREF được dùng trong Gauss-Jordan để tìm nghiệm tổng quát và ma trận nghịch đảo.

1.1.2. Khử Gauss có chọn phần tử chốt

Trong quá trình tính toán, việc chọn pivot nhỏ có thể gây sai số lớn. Do đó, tại bước k, ta chọn dòng p sao cho:

$$|a_{pk}^{(k)}| = \max_{k \leq i \leq n} |a_{ik}^{(k)}| \tag{1.1}$$

Sau đó hoán đổi dòng k và p.

Kỹ thuật này giúp giảm sai số khi tính toán, tăng tính ổn định số học và tránh việc phải chia cho số rất nhỏ.

1.1.3. Tính định thức bằng khử Gauss

Sau khi đưa ma trận về dạng tam giác trên U, định thức của U được tính bằng cách lấy tích các phần tử trên đường chéo chính.

$$det(A) = (-1)^s \prod_{i=1}^{n} u_{ii} \tag{1.2}$$

Trong đó:

- s: Số lần hoán đổi dòng.
- u_{ii}: Phần tử trên đường chéo của U.

1.1.4. Ma trận nghịch đảo (Gauss-Jordan)

Một ma trận $A \in \mathbb{R}^{n\times n}$ khả nghịch khi và chỉ khi:

$$\det(A) \neq 0.$$

Để tìm A^{-1}, ta xét ma trận ghép:

$$[A \mid I_n]$$

Sau đó áp dụng khử Gauss-Jordan để đưa về dạng:

$$[I_n \mid A^{-1}]$$

Quá trình này tương đương với việc tìm ma trận nghịch đảo thông qua các phép biến đổi dòng.

1.1.5. Hạng ma trận và các không gian con

Định nghĩa. Hạng của ma trận A, ký hiệu $\text{rank}(A)$, là số pivot trong dạng REF của A.

Từ dạng RREF, ta xác định được các không gian quan trọng:

- **Không gian cột** $C(A)$: sinh bởi các cột pivot của ma trận gốc
- **Không gian dòng** $R(A)$: sinh bởi các dòng khác 0 của RREF

- **Không gian nghiệm** $N(A)$: tập nghiệm của hệ thuần nhất $Ax = 0$

Các không gian này đóng vai trò quan trọng trong việc hiểu cấu trúc tuyến tính của ma trận và hệ phương trình.

1.2. Kết quả thực nghiệm và so sánh với Numpy

Để kiểm chứng tính đúng đắn của các thuật toán đã cài đặt, nhóm sử dụng Jupyter Notebook `part1_demo.ipynb` nhằm minh họa quá trình thực thi và so sánh kết quả với thư viện NumPy.

1.2.1. Mục tiêu

Notebook được xây dựng với các mục tiêu chính sau:

- Minh họa từng bước của phép khử Gauss với partial pivoting
- Kiểm tra nghiệm của hệ phương trình tuyến tính $Ax = b$
- Tính định thức và so sánh với `numpy.linalg.det`
- Tính ma trận nghịch đảo và kiểm chứng $AA^{-1} \approx I$
- Xác định hạng ma trận và các không gian con

1.2.2. Các trường hợp kiểm thử

Nhóm tiến hành kiểm thử trên nhiều loại ma trận khác nhau:

- Ma trận vuông khả nghịch ($\det \neq 0$)
- Ma trận suy biến ($\det = 0$)
- Ma trận có pivot nhỏ (để kiểm tra partial pivoting)
- Ma trận ngẫu nhiên kích thước nhỏ và trung bình

Với mỗi ma trận A và vector b, nhóm thực hiện:

1. Giải hệ $Ax = b$ bằng thuật toán tự cài đặt
2. So sánh nghiệm với kết quả từ NumPy
3. Tính định thức bằng hai phương pháp và so sánh sai số

4. Tính ma trận nghịch đảo và kiểm tra: $|AA^{-1} - I| \approx 0$

5. So sánh hạng ma trận với `numpy.linalg.matrix_rank`

1.2.3. Đánh giá sai số

Giải hệ phương trình

STT	Trường hợp kiểm thử	Trạng thái	Sai số
1	Ma trận 3x3 thông thường	Đã kiểm chứng	8.88×10^{-16}
2	Ma trận đơn vị 4x4 (kích thước lớn)	Đã kiểm chứng	0.00×10^{0}
3	Ma trận số thực (float)	Đã kiểm chứng	0.00×10^{0}
4	Ma trận chữ nhật (nhiều cột hơn hàng)	Đã kiểm chứng	0.00×10^{0}
5	Ma trận suy biến / có hàng 0	Đã kiểm chứng	0.00×10^{0}
6	Ma trận có hàng trùng lặp	Đã kiểm chứng	0.00×10^{0}

Bảng 1.1: Kết quả kiểm thử phương pháp Gauss

Tính toán định thức

STT	Trường hợp kiểm thử	Trạng thái	Định thức
1	Ma trận đơn vị 3x3	Đã kiểm chứng	1.0000
2	Ma trận tam giác trên	Đã kiểm chứng	30.0000
3	Ma trận suy biến (det = 0)	Đã kiểm chứng	0.0000
4	Ma trận kém điều kiện (kích thước nhỏ)	Đã kiểm chứng	0.0001
5	Ma trận ngẫu nhiên 3x3	Đã kiểm chứng	−306.0000
6	Ma trận có phần tử âm	Đã kiểm chứng	−2.0000

Bảng 1.2: Kết quả kiểm thử tính định thức

Phân tích hệ số điều kiện

Trường hợp	Vector b	Nghiệm x	Ghi chú
b_1	[2.0000, 2.0001]	[0.9999999999977796, 1.0000000000022204]	Nghiệm ổn định
b_2	[2.0000, 2.0000]	[0.0, 2.0]	Nghiệm thay đổi mạnh

Bảng 1.3: Phân tích ảnh hưởng của hệ số điều kiện

Hạng và không gian cơ sở

STT	Trường hợp kiểm thử	Hạng	Trạng thái
1	Ma trận hạng đầy đủ 3x3	3	Đã kiểm chứng
2	Ma trận hạng 1 (các hàng phụ thuộc)	1	Đã kiểm chứng
3	Ma trận không 2x2	0	Đã kiểm chứng
4	Ma trận chữ nhật 3x4	2	Đã kiểm chứng
5	Ma trận cao 4x2	2	Đã kiểm chứng

Bảng 1.4: Kết quả kiểm thử hạng ma trận và không gian nghiệm

CHƯƠNG 2

Phân Rã Ma Trận Và Trực Quan Hóa Với Manim

2.1. Phân rã giá trị suy biến (SVD)

2.1.1. Định nghĩa ma trận SVD

Phân rã giá trị suy biến - Singular Value Decomposition là một công cụ mạnh mẽ trong đại số tuyến tính, nó cho phép phân tích một ma trận bất kỳ $A \in \mathbb{R}^{m\times n}$ thành ba ma trận đặc biệt:

$$A = U\Sigma V^T \tag{2.1}$$

Trong đó:

- $U \in \mathbb{R}^{m\times m}$: Là ma trận trực giao ($U^TU = I_m$), các cột của nó là các vector suy biến trái.
- $\Sigma \in \mathbb{R}^{m\times n}$: Là ma trận đường chéo không vuông, chứa các giá trị suy biến σ_i trên đường chéo chính.
- $V^T \in \mathbb{R}^{n\times n}$: Là ma trận trực giao ($V^TV = I_n$); các vector suy biến phải là các cột của V (tương đương các hàng của V^T).

2.1.2. Tính chất của các giá trị suy biến

Thứ tự và dấu: Các giá trị suy biến luôn không âm và thường được sắp xếp giảm dần.

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_p \geq 0 \text{ với } p = \min(m, n) \tag{2.2}$$

Hạng ma trận: Số lượng các giá trị suy biến khác 0 tương ứng với hạng của ma trận A.

Xấp xỉ hạng thấp: SVD cung cấp xấp xỉ tốt nhất hạng k qua công thức $A_k = \sum_{i=1}^{k} \sigma_i u_i v_i^T$, ứng dụng trong nén ảnh và khử nhiễu.

2.1.3. Mối liên hệ với giá trị riêng (Eigenvalues)

SVD là sự tổng quát hóa của phép chéo hóa, có mối quan hệ chặt chẽ với các giá trị riêng của ma trận đối xứng:

- **Công thức liên hệ**: Các giá trị suy biến σ_i của A là căn bậc hai của các giá trị riêng λ_i của ma trận A^TA:

$$\sigma_i = \sqrt{\lambda_i(A^T A)} \tag{2.3}$$

- **Cơ sở vector**: Các cột của V là các vector riêng của $A^T A$, trong khi các cột của U là các vector riêng của AA^T.

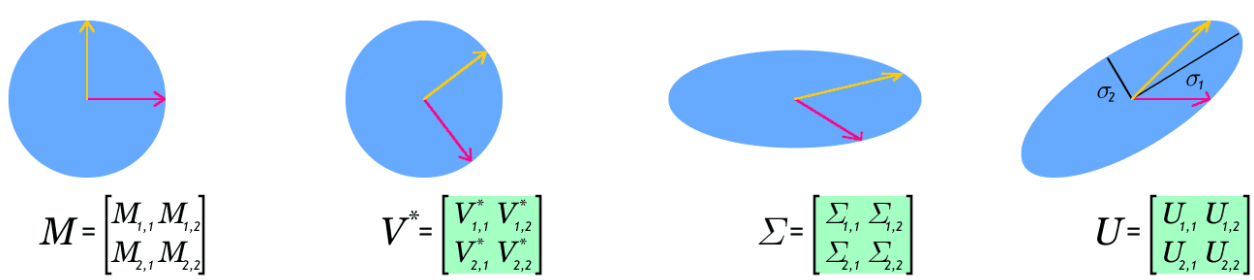
2.2. Ý nghĩa của phân rã SVD

2.2.1. Cơ chế hoạt động

SVD không chỉ đơn thuần là một công thức đại số, về mặt bản chất nó chính là một chuỗi các phép biến đổi hình học cơ bản. Bất kỳ ma trận $A \in \mathbb{R}^{m \times n}$ nào cũng biểu diễn một phép biến đổi tuyến tính từ không gian $\mathbb{R}^n$ sang không gian $\mathbb{R}^m$.

SVD cho phép phân tách phép biến đổi này thành ba bước độc lập:

- Phép xoay đầu tiên (V^T): Ma trận trực giao V^T thực hiện xoay hệ tọa độ trong không gian sao cho các trục tọa độ ban đầu trùng với các vector suy biến phải (v_i). Bước này không làm thay đổi độ dài của các vector.
- Phép co giãn (Σ): Ma trận đường chéo Σ thực hiện co giãn theo không gian dọc theo các trục tọa độ mới. Các vector sẽ được co giãn hoặc rút ngắn theo các hệ số σ_i. Nếu $\sigma_i = 0$, chiều không gian tương ứng bị triệt tiêu, điều đó giải thích vì sao mà SVD có thể xác định được hạng (rank) và không gian null (nullspace).
- Phép xoay thứ hai (U): Ma trận trực giao U xoay hệ tọa độ một lần nữa trong không gian để đưa các vector về vị trí cuối cùng theo các vector suy biến trái.



Hình 2.1: Các bước phân rã SVD

2.2.2. Vì sao cơ chế này quan trọng?

Trong khi phép chéo hóa $A = PDP^{-1}$ chỉ hoạt động khi ta có thể tìm các trục mà tại đó ma trận chỉ thực hiện co giãn thì SVD chỉ ra rằng mọi ma trận (kể cả ma trận hình chữ nhật) đều có thể tìm được một cặp cơ sở trực chuẩn (V và U) để biến hình cầu thành một hình ellipse.

Vì U và V là các ma trận trực giao, chúng không làm khuếch đại sai số tính toán làm cho thuật toán này trở nên ổn định nhất trong đại số tuyến tính.

2.3. Thuật toán giá trị riêng Jacobi

2.3.1. Định lý bất khả Abel

Định lý phát biểu rằng **không tồn tại nghiệm đại số** - tức là nghiệm được biểu diễn bằng căn thức của phương trình đa thức tổng quát **bậc 5 hoặc lớn hơn** với các hệ số bất kỳ.

Trong quá trình tiến hành thuật toán SVD, chúng ta cần phải tìm các trị riêng của ma trận đối xứng A^TA. Việc tìm giá trị riêng tương đương với việc tìm nghiệm của đa thức đặc trưng $P(\lambda) = \det(A^TA - \lambda I) = 0$ nên đối với các ma trận có kích thước ≥ 5, chúng ta không thể sử dụng các phương pháp đại số thông thường để giải quyết bài toán trên.

Dựa trên giới hạn của định lý Abel, ta phải sử dụng các phương pháp lặp số trị để giải quyết bài toán trên. Thuật toán Jacobi là lựa chọn tối ưu cho ma trận đối xứng vì:

- Nó triệt tiêu các phần tử ngoài đường chéo bằng các phép xoay cho đến khi ma trận dần hội tụ về dạng đường chéo.
- Đảm bảo tính số học cao, phù hợp với yêu cầu ổn định số học của đồ án.

2.3.2. Chi tiết thuật toán

Phép xoay Jacobi hai phía (Two-sided Jacobi)[1]

Tại mỗi bước lặp, thuật toán chọn một phần tử ngoài đường chéo s_{ij} (thường là phần tử có giá trị tuyệt đối lớn nhất) và triệt tiêu nó về 0 bằng một phép xoay trực giao, gọi là **ma trận xoay Jacobi** $G(i, j, \theta)$.

Ma trận G giống với ma trận đơn vị nhưng có phần tử thay đổi tại bốn vị trí: g_{ii}, g_{jj}, g_{ij} và g_{ji}. Khi đó, ma trận được cập nhật theo công thức: $S' = G^T SG$.

$$G(i,j,\theta) = \begin{bmatrix} 1 & & & \text{Cột } i & & \text{Cột } j & & \\ & \ddots & & \downarrow & & \downarrow & & \\ & & 1 & & & & & \\ & & & \cos\theta & \dots & -\sin\theta & \to \text{Dòng } i & \\ & & & \vdots & 1 & \vdots & & \\ & & & \sin\theta & \dots & \cos\theta & \to \text{Dòng } j & \\ & & & & & 1 & & \\ & & & & & & \ddots & \\ & & & & & & & 1 \end{bmatrix} \tag{2.4}$$

Vì G là ma trận trực giao nên phép biến đổi trên giữ nguyên đa thức đặc trưng của ma trận. Tức là S và S' có cùng tập hợp các trị riêng. Do đó mà ta có thể lặp lại quá trình mà vẫn giữ được các trị riêng ban đầu của $A^T A$.

Không những thế, ma trận trực giao G có đặc tính bảo toàn bình phương tất cả các phần tử. Khi phép quay làm một phần tử ngoài đường chéo triệt tiêu về 0, giá trị đó không biến mất mà được "đẩy" vào các phần tử trên đường chéo chính.

Tính toán góc xoay θ

Để phần tử s'_{ij} trở về bằng 0 sau phép biến đổi, góc θ được xác định dựa trên các phần tử hiện tại của ma trận S:

- Nếu $s_{ii} = s_{jj}$ thì $\theta = \pi/4$.
- Nếu $sii \neq s_{jj}$, ta tính θ thông qua giá trị $\tau = \frac{s_{jj} - s_{ii}}{2s_{ij}}$, sau đó tìm $t = tan(\theta)$ là nghiệm của phương trình $t^2 + 2rt - 1 = 0$.

[1] Ngoài ra còn tồn tại phương pháp Jacobi một phía với nguyên lý tương tự.

Các bước thực hiện thuật toán

Algorithm 1 THUẬT TOÁN TRỊ RIÊNG JACOBI

```
procedure JACOBI_EIGENVALUES(A, ϵ, MAX_ITER)
    S ← A^T A, V ← I_n
    iter ← 0
    while iter < max_iter do
        max_val ← max_{i≠j} |s_ij|
        if max_val < ϵ then
            break
        end if
        cos(θ), sin(θ) được tính từ s_ii, s_jj, s_ij
        S ← G^T S G (ma trận S dần trở thành ma trận đường chéo Σ^2)
        V ← VG (các cột của V dần trở thành vector suy biến phải)
        iter ← iter + 1
    end while
    eigenvalues ← Các phần tử trên đường chéo chính của S
    return eigenvalues, V
end procedure
```

Ghi chú:

- Tại mỗi bước lặp, việc lựa chọn cặp chỉ số (i, j) có thể dựa trên phần tử ngoài đường chéo có giá trị tuyệt đối lớn nhất nhằm tăng tốc độ hội tụ.
- Góc quay θ được tính toán sao cho triệt tiêu phần tử s_{ij}, đồng thời đảm bảo ma trận quay là trực giao.
- Các phép quay cần được tích lũy qua các bước lặp để xây dựng các ma trận trực giao cuối cùng.
- Thuật toán dừng khi tất cả các phần tử ngoài đường chéo đủ nhỏ (theo ngưỡng ε cho trước).
- Trong thực tế, có thể cần nhiều vòng lặp qua toàn bộ các cặp chỉ số để đạt được sự hội tụ.

2.4. Chéo hóa ma trận

2.4.1. Cơ sở lý thuyết

Một ma trận vuông $A \in \mathbb{R}^{n\times n}$ được gọi là chéo hóa được nếu tồn tại một ma trận khả nghịch P và một ma trận đường chéo D sao cho:

$$A = PDP^{-1} \tag{2.5}$$

Trong đó:

- D là ma trận đường chéo $\text{diag}(\lambda_1, \lambda_2, \ldots, \lambda_n)$, với λ_i là các giá trị riêng của A.
- P là ma trận các vector riêng, trong đó mỗi cột của P là một vector riêng ứng với giá trị riêng λ_i.

Quá trình chéo hóa phụ thuộc hoàn toàn vào việc tìm kiếm các đặc trưng phổ của ma trận:

- **Trị riêng** λ: Trị riêng được xác định từ phương trình đặc trưng $det(A - \lambda I) = 0$. Đây là giá trị vô hướng thể hiện mức độ co giãn của ma trận theo một hướng nhất định.
- **Vector riêng** x: Vector riêng tương ứng là nghiệm của hệ $(A - \lambda I)x = 0$. Nó đại diện cho hướng mà tại đó phép biến đổi tuyến tính chỉ đóng vai trò như phép nhân vô hướng.
- **Điều kiện chéo hóa**: Ma trận $A \in \mathbb{R}^{n\times n}$ chéo hóa được khi và chỉ khi tồn tại đủ n vector riêng độc lập tuyến tính để cấu thành các cột của ma trận khả nghịch P.

2.4.2. Trường hợp đặc biệt: Ma trận đối xứng thực

Định lý phổ (Spectral Theorem)

Đối với ma trận vuông bất kỳ, việc chéo hóa được hay không phụ thuộc vào số lượng vector riêng độc lập tuyến tính. Tuy nhiên đối với ma trận đối xứng thực A, định lý phổ khẳng định:

- A **luôn** chéo hóa được.
- Tất cả các trị riêng của A đều là **số thực**.
- Các vector riêng ứng với các giá trị riêng khác nhau của A luôn **trực giao** với nhau.

Chéo hóa trực giao

Vì các vector riêng của ma trận đối xứng thực có thể chọn để tạo thành một bộ cơ sở trực chuẩn, công thức chéo hóa trở thành:

$$A = QDQ^{-1} \tag{2.6}$$

Trong đó:

- Q là ma trận trực giao thỏa mãn $Q^{-1} = Q^T$. Điều này giúp việc tính toán ma trận nghịch đảo trở nên đơn giản bằng một phép chuyển vị thay vì tốn chi phí $O(n^3)$ thông thường với ma trận P.
- D là ma trận đường chéo chứa các trị riêng của A.

Vai trò và ý nghĩa trong tính toán

- **Sự ổn định**: Các phép biến đổi trực giao không làm khuếch đại sai số làm tròn, đảm bảo độ chính xác cực kỳ cao ($< 10^{-10}$) khi xử lý trong thực tế.
- **Mối liên hệ với SVD**: Đây là sự kết nối trực tiếp đến phân rã SVD. Trong SVD, chúng ta phải thực hiện chéo hóa ma trận đối xứng A^TA để tìm các giá trị suy biến và vector suy biến phải.
- **Tối ưu hiệu năng**: Đối với ma trận đối xứng xác định dương, chi phí tính toán có thể giảm xuống bằng một nửa so với các phương pháp thông thường.

2.5. Phương pháp tìm trị riêng và vector riêng

2.5.1. Thuật toán QR với kỹ thuật Deflation

Thuật toán QR là một phương pháp nhằm đưa ma trận A về dạng tam giác trên (dạng Schur). Khi ma trận A là ma trận tam giác trên, các trị riêng chính là các phần tử nằm trên đường chéo chính.

Các bước thực hiện

1. **Khởi tạo:** Đặt $A_0 = A$.
2. **Lặp (Iteration):** Tại mỗi bước k, thực hiện phân rã ma trận:

$$A_k = Q_k R_k$$

Sau đó cập nhật ma trận cho bước kế tiếp bằng cách nhân ngược theo thứ tự:

$$A_{k+1} = R_k Q_k$$

Ghi chú: A_{k+1} đồng dạng với A_k ($A_{k+1} = Q_k^{-1} A_k Q_k$), nên chúng có cùng tập trị riêng.

3. **Hội tụ:** Qua nhiều vòng lặp, các phần tử phía dưới đường chéo chính tiến dần về số 0.

Kỹ thuật Deflation

Để tối ưu tốc độ của thuật toán, sau khi phần tử $A[n-1][n-2]$ đủ nhỏ, tức là $A[n-1][n-2] < \epsilon$ với ϵ là ngưỡng hội tụ cho phép:

- Ta coi phần tử ở góc cuối $A[n-1][n-1]$ đã hội tụ thành một trị riêng.
- Thu nhỏ ma trận bằng cách cắt bỏ hàng cuối và cột cuối (giảm kích thước từ n xuống $n-1$)
- Tiếp tục quy trình đến khi ma trận chỉ còn kích thước 1×1.

2.5.2. Dịch chuyển Wilkinson (Wilkinson Shift)

Wilkinson shift được sử dụng để tăng tốc độ hội tụ của thuật toán QR. Bằng cách chọn một giá trị dịch chuyển gần với trị riêng thực của ma trận (ước lượng từ khối ma trận 2×2 cuối), thuật toán làm giảm nhanh các phần tử đường chéo, giúp quá trình deflation diễn ra nhanh hơn giúp cải thiện hiệu năng đáng kể.

Công thức dịch chuyển

Thay vì phân rã trực tiếp A_k, ta thực hiện:

$$A_k - \mu I = Q_k R_k \qquad A_{k+1} = R_k Q_k + \mu I \tag{2.7}$$

Tính giá trị Wilkinson Shift

Giá trị μ được chọn là trị riêng của khối con 2×2 ở góc dưới bên phải ma trận, cụ thể là nghiệm gần với phần tử đường chéo cuối cùng nhất.

Xét khối 2×2 cuối:

$$B = \begin{pmatrix} a & b \\ c & d \end{pmatrix} = \begin{pmatrix} A_{n-2,n-2} & A_{n-2,n-1} \\ A_{n-1,n-2} & A_{n-1,n-1} \end{pmatrix}$$

1. **Tìm nghiệm**: Giải phương trình đặc trưng của B:

$$\lambda^2 - tr(B)\lambda + det(B) = 0 \quad \Rightarrow \quad \lambda = \frac{(a+d) \pm \sqrt{(a+d)^2 - 4(ad-bc)}}{2}$$

2. **Chọn** μ: μ là trị riêng gần với phần tử đường chéo cuối cùng nhất, tức là trị riêng có giá trị tuyệt đối nhỏ nhất trong hai nghiệm.

2.5.3. Phân rã QR bằng Gram-Schmidt cải tiến

Trong môi trường dấu phẩy động, thuật toán Gram-Schmidt cổ điển dễ làm mất đi tính trực giao giữa các vector do sai số làm tròn trong quá trình tích lũy. Cụ thể hơn, nguyên nhân nằm ở các phép trừ liên tiếp giữa các vector gần song song theo công thức:

$$v_j \leftarrow v_j - \sum_{i=1}^{k} (q_i^T v_j) q_i \tag{2.8}$$

Nếu Q không trực giao đủ tốt sẽ dẫn đến $Q^T Q \neq I$ và làm cho lặp QR bị sai và cuối cùng các trị riêng bị sai hoặc hội tụ chậm.

Đối với thuật toán Gram-Schmidt cải tiến, việc trực giao hóa từng bước sẽ được thực hiện ngay lập tức ngay sau khi có được q_i

$$v_j \leftarrow v_j - (q_i^T v_j) q_i \tag{2.9}$$

sẽ giúp v_j cập nhật liên tục và sẽ làm giảm sai số tích lũy. Do đó mà Gram-Schmidt cải tiến thường được sử dụng phổ biến hơn trong thực tế so với cách thông thường.

Algorithm 2 THUẬT TOÁN GRAM-SCHMIDT CẢI TIẾN

```
procedure QR_DECOMPOSITION(A)
    n ← số cột của A
    Q ← 0_{n×n},  R ← 0_{n×n}
    V ← các cột của A
    for i = 1 to n do
        R_ii ← ||V_i||_2
        if R_ii < ϵ then
            q_i ← vector đơn vị trực giao với các cột trước đó của Q
            R_ii ← 0
        else
            q_i ← V_i / R_ii
        end if
        Gán q_i làm cột thứ i của Q
        for j = i + 1 to n do
            R_ij ← q_i^T V_j
            V_j ← V_j − R_ij q_i
        end for
    end for
    return Q, R
end procedure
```

Điều kiện dừng và hội tụ:

- Ngưỡng sai số: $\epsilon = 1e - 10$
- Deflation khi: $|A[n-1][n-2]| < \epsilon$
- Số vòng lặp tối đa: $MAX_ITERATION = 3000$
  Nếu không hội tụ sau số vòng lặp này, thuật toán sẽ báo lỗi.

2.6. Kết quả

A	**U**	**Σ**	**V^T**	**$U\Sigma V^T$**
$\begin{bmatrix} -1.0 & 3.0 \\ -2.0 & 4.0 \end{bmatrix}$	$\begin{bmatrix} 0.576 & 0.8174 \\ 0.8174 & -0.576 \end{bmatrix}$	$\begin{bmatrix} 5.465 & 0.0 \\ 0.0 & 0.366 \end{bmatrix}$	$\begin{bmatrix} -0.4046 & 0.9145 \\ 0.9145 & 0.4046 \end{bmatrix}$	$\begin{bmatrix} -1.0 & 3.0 \\ -2.0 & 4.0 \end{bmatrix}$
$\begin{bmatrix} 1.0 & 2.0 & 3.0 \\ 4.0 & 5.0 & 6.0 \\ 7.0 & 8.0 & 9.0 \end{bmatrix}$	$\begin{bmatrix} 0.2148 & -0.8872 & 0.8981 \\ 0.5206 & -0.2496 & 0.4082 \\ 0.8263 & 0.3879 & -0.1633 \end{bmatrix}$	$\begin{bmatrix} 16.8481 & 0.0 & 0.0 \\ 0.0 & 1.0684 & 0.0 \\ 0.0 & 0.0 & 0.0 \end{bmatrix}$	$\begin{bmatrix} 0.4797 & 0.5724 & 0.6651 \\ 0.7767 & 0.0757 & -0.6253 \\ 0.4082 & -0.8165 & 0.4082 \end{bmatrix}$	$\begin{bmatrix} 1.0 & 2.0 & 3.0 \\ 4.0 & 5.0 & 6.0 \\ 7.0 & 8.0 & 9.0 \end{bmatrix}$
$\begin{bmatrix} 1 & 1 & 1 \\ 2 & 2 & 2 \\ 3 & 3 & 3 \end{bmatrix}$	$\begin{bmatrix} 0.2673 & -0.3333 & 0.0 \\ 0.5345 & -0.6667 & 0.0 \\ 0.8018 & -0.6667 & 0.0 \end{bmatrix}$	$\begin{bmatrix} 6.4807 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	$\begin{bmatrix} 0.5774 & 0.5774 & 0.5774 \\ 0.4082 & 0.4082 & -0.8165 \\ 0.7071 & -0.7071 & 0 \end{bmatrix}$	$\begin{bmatrix} 1 & 1 & 1 \\ 2 & 2 & 2 \\ 3 & 3 & 3 \end{bmatrix}$
$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$
$\begin{bmatrix} 5 & 4 & 4 \\ 0 & 2 & 1 \\ 0 & 0 & 0.5 \end{bmatrix}$	$\begin{bmatrix} 0.9761 & 0.2154 & -0.028 \\ 0.2144 & -0.9762 & -0.0339 \\ 0.0346 & -0.0271 & 0.999 \end{bmatrix}$	$\begin{bmatrix} 7.727 & 0 & 0 \\ 0 & 1.5382 & 0 \\ 0 & 0 & 0.4207 \end{bmatrix}$	$\begin{bmatrix} 0.6316 & 0.5608 & 0.5353 \\ 0.7002 & -0.7091 & -0.0833 \\ -0.3329 & -0.4274 & 0.8405 \end{bmatrix}$	$\begin{bmatrix} 5 & 4 & 4 \\ 0 & 2 & 1 \\ 0 & 0 & 0.5 \end{bmatrix}$

Bảng 2.1: Kết quả phân rã SVD cho các ma trận kiểm thử

2.7. Trực quan hóa với Manim

2.7.1. Nội dung của video

Video tập trung phân tích và mô phỏng lý thuyết **phân rã giá trị suy biến (Singular Value Decomposition - SVD)**, một phương pháp chuẩn hóa ma trận có tính tổng quát cao, khắc phục giới hạn tồn tại của phương pháp chéo hóa (Diagonalization) truyền thống. Nội dung video được cấu trúc thành ba phần chính:

- **Phân tích hình học và đại số**: Minh họa quy trình phân tách một phép biến đổi tuyến tính thành tích của ba ma trận thành phần: Phép xoay trong không gian nguồn thông qua ma trận đơn nhất V^T. Phép co giãn hình học (scaling) dựa trên ma trận đường chéo các giá trị suy biến Σ. Phép xoay trong không gian đích thông qua ma trận đơn nhất U.
- **So sánh tính ưu việt**: Phân tích lý do vì sao SVD được coi là phương pháp phân rã tối ưu trong việc xử lý các ma trận không vuông hoặc không đầy đủ hạng bằng cách sử dụng hệ hai cơ sở trực chuẩn khác biệt (U và V), thay vì sử dụng một hệ cơ sở trực chuẩn duy nhất như trong chéo hóa ma trận.
- **Ứng dụng thực tiễn**: Trình bày nguyên lý hoạt động của SVD như một công cụ lọc nhiễu và nén thông tin thông qua việc xấp xỉ ma trận hạng thấp. Các ví dụ thực tiễn bao gồm: nén dữ liệu hình ảnh, phân tích thành phần chính (PCA) trong thống kê, phân tích ngữ nghĩa ẩn (Latent Semantic Analysis - LSA)...

2.7.2. Động lực thực hiện

Động lực chính của đồ án xuất phát từ việc tối ưu hóa khả năng tiếp cận và nhận thức đối với một trong những khái niệm nền tảng phức tạp nhất của Đại số tuyến tính:

- **Trực qua hóa sự trừu tượng**: Trong môi trường học thuật, các phép phân rã chỉ thường được tiếp cận thông qua các hệ thống công thức và thuật toán đại số thuần túy mà bỏ quên đi ý tưởng hình học đằng sau đó. Đồ án sử dụng thư viện Manim để chuyển đổi các định nghĩa trừu tượng sang các thực thể hình học, giúp tăng sự trực quan của người xem thông qua các phép biến đổi trong không gian.

- **Giải quyết giới hạn của phương pháp truyền thống**: Nhằm làm rõ tính ưu việt của SVD so với phương pháp chéo hóa thông thường. Việc trực quan hóa giúp minh chứng rằng SVD là công cụ tổng quát hơn, giải quyết được nhiều trường hợp hơn mà không bị ràng buộc bởi tính chất, đặc trưng của các ma trận vuông hoặc hệ vector riêng.
- **Kết nối lý thuyết với các bài toán thực tiễn hiện đại**: SVD không chỉ là khái niệm lý thuyết mà còn là nhân tố cốt lõi trong các công nghệ đột phá như nén dữ liệu, giảm chiều dữ liệu (PCA) và xử lý ngôn ngữ tự nhiên (LSA). Đồ án nhằm xây dựng một cái nhìn toàn diện, giúp người học hiểu rõ tại sao các phép hình học cơ bản lại ứng dụng trong các hệ thống AI và phân tích dữ liệu lớn hiện nay.

2.7.3. Quy trình xây dựng video

Kịch bản video

Nội dung video được xây dựng theo hướng đi từ cụ thể đến trừu tượng. Mở đầu bằng việc so sánh những sự hạn chế đối với phương pháp chéo hóa thông thường và tổng quát hóa sang SVD. Sau đó phân tích ý nghĩa hình học của SVD thông qua việc mô hình hóa các công thức toán học thành các đối tượng hình học trong không gian. Cuối cùng, liệt kê ra các ứng dụng của SVD trong thực tiễn nhằm nhấn mạnh tầm quan trọng của nó.

Ngoài ra, nội dung thuyết mình còn được chuẩn hóa và ánh xạ trực tiếp sang bảng điều phối (**cue sheet**). Tài liệu trên đóng vai trò như trục thời gian tham chiếu, giúp đồng bộ chính xác các hàm xuất hiện trong mã nguồn.

Mã nguồn

Đồ án sử dụng thư viện **Manim** để lập trình các thực thể toán học. Các quy tắc diễn hoạt (**animation rules**) được thiết lập để đảm bảo tính nhất quán về thẩm mỹ (màu sắc, tốc độ chuyển động, độ dày nét vẽ...) xuyên suốt toàn bộ video.

Thuyết minh

Phần âm thanh được thu âm dựa trên kịch bản đã điều chỉnh, chú trọng vào việc điều phối nhịp độ trùng khớp với các chuyển động hoạt ảnh trên màn hình, giúp người xem dễ dàng theo dõi các lập luận logic toán học.

Chỉnh sửa video

Sử dụng các kỹ thuật cắt ghép và hiệu ứng chuyển cảnh để nối các scene lại với nhau trong các phân đoạn mô phỏng. Giai đoạn này tập trung làm nổi bật các mối quan hệ trọng yếu (như sự tương quan giữa các giá trị đơn lẻ và độ dài bán trục ellipse)

2.7.4. Phân tích kỹ thuật chuyên sâu

So sánh vector riêng và vector suy biến phải

Một trong những điều gây nhầm lẫn là sự khác biệt giữa vector riêng (eigenvectors) và vector suy biến phải (right singular vectors - cột của V). Trong quá trình trực quan hóa, nhóm đã hiển thị đồng thời cả **các vector riêng của** A và **các vector** V **của SVD**.

Sau khi áp dụng các phép biến đổi tuyến tính lên A:

- **Vector riêng**:
  - ❏ Không thay đổi phương.
  - ❏ Chỉ bị co giãn theo hệ số λ (là các trị riêng)
- **Vector suy biến**:
  - ❏ Bị thay đổi phương sau khi biến đổi.
  - ❏ Tuy nhiên luôn duy trì tính **trực giao**.

Điều này nhấn mạnh sự khác biệt quan trọng: Vector riêng đặc trưng cho cấu trúc nội tại của A, trong khi vector suy biến mô tả cách A biến đổi toàn bộ không gian theo một hệ trục trực chuẩn tối ưu.

Trực quan hóa Jacobi hai phía

Phần này minh họa trực tiếp cơ chế của thuật toán Jacobi hai phía (Two-sided Jacobi) thông qua các bước hình học.

- **Bước 1**: Hiển thị các vector cơ sở chuẩn (e_1, e_2) và các vector V (khởi đầu là trực chuẩn)
- **Bước 2**: Sau khi áp dụng phép biến đổi A
  - ❏ Các vector cơ sở bị biến dạng "tùy ý" (không còn trực giao so với ban đầu)

- ❏ Các vector V vẫn duy trì được sự trực giao, dần xây dựng hệ thống trục trực chuẩn ổn định.

- **Bước 3**: Áp dụng thêm phép biến đổi A^T, hệ thu được là A^TA. Khi đó:
  - ❏ Các vector cơ sở trở nên trực giao và có thể bị đảo chiều so với V.
  - ❏ Độ dài bị co giãn theo: $\lambda = \sigma^2$.
- **Bước 4**: Căn chỉnh
  - ❏ Vector cơ sở đã trực giao nhưng chưa "thẳng hàng" so với hệ trục chính.
  - ❏ Cuối cùng, thực hiện một phép xoay để thu được hệ trục trùng với vector V.

CHƯƠNG 3

Giải Hệ Phương Trình Và Phân Tích Hiệu Năng

3.1. Cơ sở lý thuyết

Phân tích hiệu năng của các thuật toán giải hệ $Ax = b$ được xây dựng trên sự tương tác giữa đặc tính cấu trúc của ma trận và cơ chế tính toán của từng phương pháp.

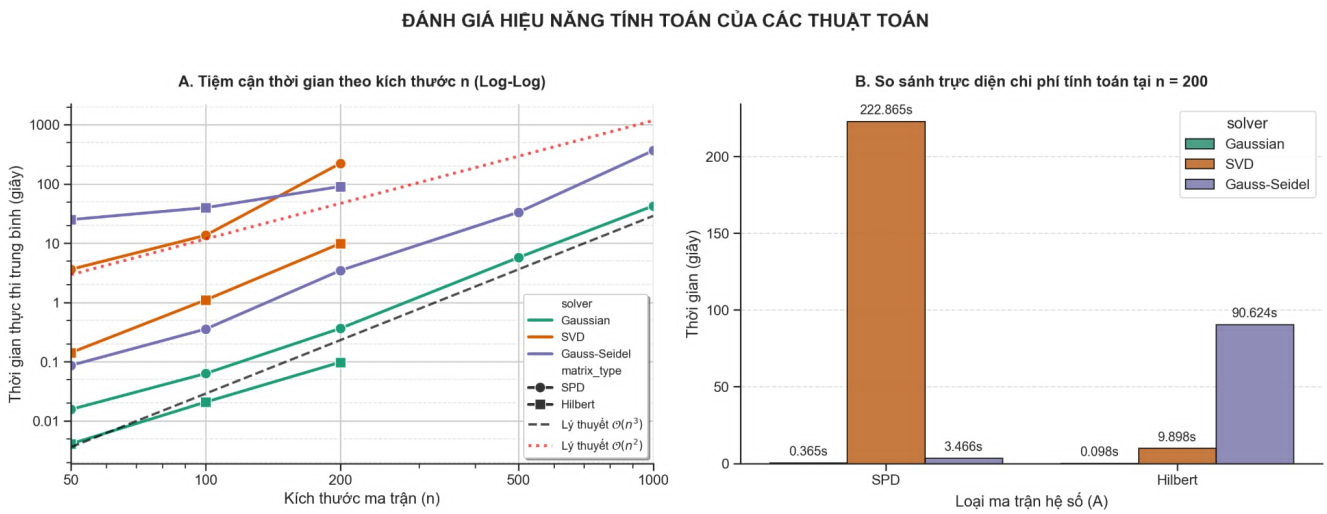
- **Ma trận Đối xứng Xác định Dương (SPD)**: Có các trị riêng thực dương ($\lambda_i > 0$). Các hệ này thường có số điều kiện (κ) thấp, giúp kiểm soát tốt sự lan truyền sai số và đảm bảo tính hội tụ cho các phương pháp lặp.
- **Ma trận Hilbert**: Là ví dụ điển hình của hệ ma trận xấu (ill-conditioned). Các trị riêng phân rã về 0 theo hàm mũ khiến định thức tiệm cận 0.
- **Số điều kiện ($\kappa(A)$)**: Thước đo độ nhạy hệ thống (với SPD, $\kappa = \lambda_{max}/\lambda_{min}$). Theo định lý nhiễu loạn, sai số nghiệm tỷ lệ thuận với κ. Trên hệ Hilbert, κ bùng nổ vượt quá giới hạn biểu diễn của số thực 64-bit, dẫn đến thất bại trong tính toán.

Các bộ giải được chia thành hai trường phái, đối mặt với sự suy biến theo những cách khác biệt:

- **Nhóm Phương pháp Trực tiếp (Direct Methods)** Đặc trưng bởi tính tất định, giải bài toán trong số bước $\mathcal{O}(N^3)$ hữu hạn, nhưng phải chịu sai số số học:
  - ❑ Khử Gauss ($A = LU$): Tối ưu hóa tốc độ phần cứng cực tốt nhưng sử dụng các biến đổi không trực giao. Rất dễ bị phá hủy bởi hiện tượng sụp đổ phần tử trục (pivot collapse) và khuếch đại nhiễu trên ma trận xấu.
  - ❑ Phân rã SVD ($A = U\Sigma V^T$): Tiêu chuẩn vàng về độ ổn định ngược. Các phép biến đổi trực giao đẳng cự ($\kappa = 1$) giúp SVD không bao giờ tự sinh/khuếch đại rác số học. Điểm yếu là chi phí tính toán siêu việt khổng lồ và dễ mất toàn bộ tính ổn định nếu lập trình ngây thơ qua bước nhân hiệp phương sai $A^T A$.
- **Phương pháp Lặp (Iterative Methods) Gauss-Seidel**: Khả năng hội tụ bị quyết định tuyệt đối bởi bán kính phổ của ma trận lặp ($\rho < 1$). Trên hệ SPD lý tưởng, thuật toán tạo ra cơ chế nén sai số chủ động. Tuy nhiên, trên hệ suy biến như Hilbert, do $\rho \to 1$, động năng tiệm cận bị triệt tiêu hoàn toàn, ép hệ thống rơi vào trạng thái đình trệ giải tích (stagnation) vĩnh viễn.

3.2. Phân tích hiệu năng

Qua thực nghiệm trên hai lớp bài toán đại diện (Ma trận xác định dương SPD và ma trận Hilbert), hiệu năng của ba phương pháp giải hệ tuyến tính thể hiện rõ sự đánh đổi giữa tốc độ tính toán và độ ổn định số:



Hình 3.1: Thời gian thực thi

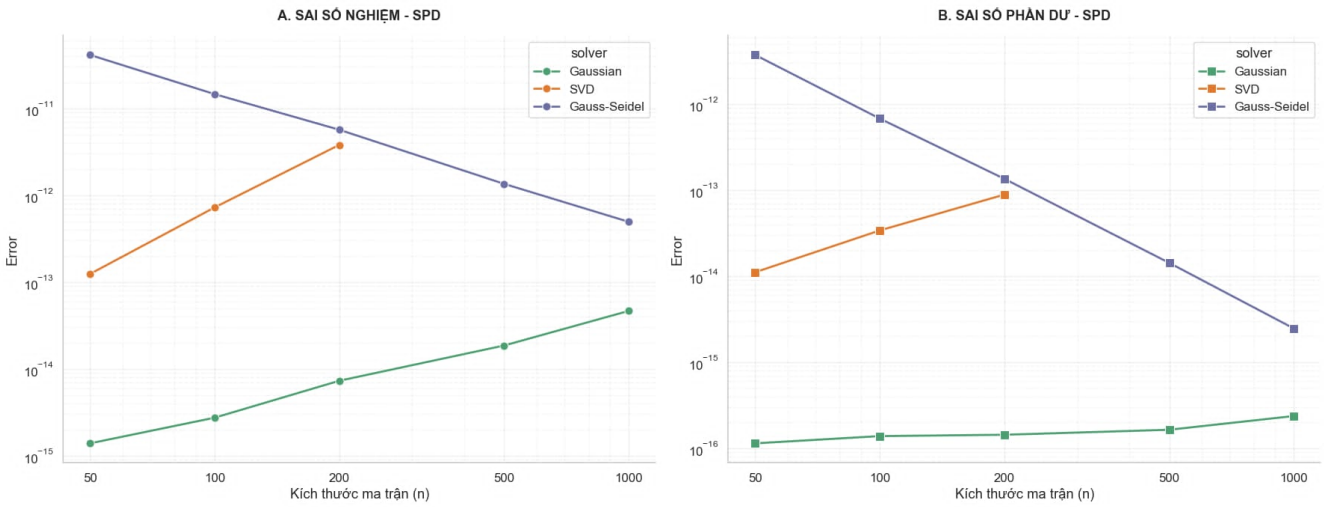
- **Khử Gauss**: Đạt thời gian thực thi thấp nhờ cấu trúc tính toán tối ưu, nhưng kém ổn định trên các bài toán điều kiện kém. Sai số làm tròn tích lũy có thể gây mất ổn định nghiêm trọng, đặc biệt khi các phần tử trục nhỏ.
- **SVD**: Có độ ổn định số cao nhờ các phép biến đổi trực giao, phù hợp với các hệ kém điều kiện. Tuy nhiên, chi phí tính toán lớn và độ nhạy với cách cài đặt (ví dụ thông qua $A^T A$) có thể làm suy giảm hiệu quả trong thực tế.
- **Gauss-Seidel**: Hiệu quả trên các hệ hội tụ tốt (như SPD), nhưng phụ thuộc mạnh vào bán kính phổ của ma trận lặp. Khi hệ gần suy biến, tốc độ hội tụ giảm đáng kể và tiêu chuẩn dừng có thể dẫn đến nghiệm không chính xác.

Kết quả cho thấy không tồn tại phương pháp tối ưu cho mọi bài toán. Các phương pháp trực tiếp chịu ảnh hưởng của sai số làm tròn, trong khi các phương pháp lặp phụ thuộc vào đặc tính phổ của ma trận. Do đó, việc lựa chọn thuật toán cần dựa trên sự cân nhắc giữa chi phí tính toán, độ ổn định số và số điều kiện $\kappa(A)$ của bài toán.

3.3. Phân tích độ ổn định

3.3.1. Hệ ma trận SPD (well-conditioned)

Để đánh giá đặc tính ổn định số, các phương pháp được khảo sát trên lớp bài toán SPD - một môi trường thuận lợi với số điều kiện thấp và đảm bảo hội tụ. Trong bối cảnh này, sai số chủ yếu phản ánh ảnh hưởng của làm tròn số và cấu trúc thuật toán, thay vì bất ổn nội tại của bài toán.



Hình 3.2: Sai số trên hệ SPD

- **Khử Gauss**: Hoạt động ổn định trên hệ SPD nhờ tránh được các vấn đề về phần tử trục. Sai số đạt gần giới hạn của độ chính xác máy (machine epsilon), cho thấy hiệu quả cao cả về độ chính xác lẫn chi phí tính toán.
- **SVD**: Đảm bảo ổn định số nhờ các phép biến đổi trực giao, tuy nhiên sai số thực nghiệm cao hơn so với Khử Gauss. Nguyên nhân chủ yếu đến từ chi phí tính toán lớn và khả năng suy hao thông tin khi triển khai thông qua các phép biến đổi trung gian như $A^T A$.
- **Gauss-Seidel**: Hội tụ tốt trên hệ SPD, nhưng độ chính xác cuối cùng phụ thuộc vào tiêu chuẩn dừng. Phương pháp thường dừng tại mức sai số phù hợp với ngưỡng dung sai đặt trước, thay vì đạt đến giới hạn chính xác của máy.

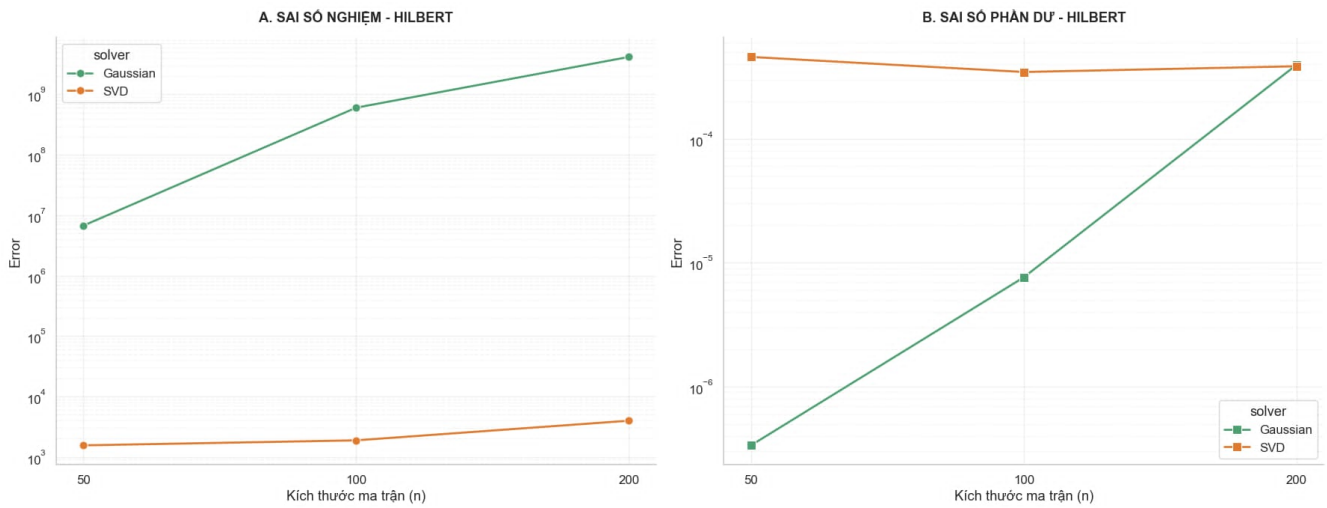
Kết quả cho thấy trong môi trường điều kiện tốt, sự khác biệt về độ ổn định giữa các phương pháp không còn mang tính quyết định. Thay vào đó, hiệu năng phụ thuộc nhiều hơn vào đặc điểm

tính toán và tiêu chí dừng, trong đó Khử Gauss đạt độ chính xác cao nhất, còn Gauss-Seidel phản ánh sự cân bằng giữa độ chính xác và chi phí tính toán.

3.3.2. Hệ ma trận Hillbert (ill-conditioned)

Phương pháp Trực tiếp (Direct Methods)

Các phương pháp trực tiếp, tiêu biểu là Khử Gauss và SVD, dựa trên việc phân rã ma trận để tìm nghiệm chính xác trong số bước hữu hạn. Tuy nhiên, trên các bài toán điều kiện kém như ma trận Hilbert, hiệu năng của chúng bị chi phối mạnh bởi sai số làm tròn và số điều kiện lớn.



Hình 3.3: Sai số trên hệ Hillbert

- **Khử Gauss**: Dễ bị mất ổn định số trên ma trận Hilbert do các phần tử trục rất nhỏ. Sai số làm tròn bị khuếch đại qua quá trình khử, dẫn đến nghiệm và phần dư sai lệch đáng kể, ngay cả khi áp dụng các kỹ thuật pivoting cơ bản.
- **SVD**: Về lý thuyết có độ ổn định cao nhờ các phép biến đổi trực giao, ít nhạy cảm với điều kiện kém. Tuy nhiên, trong thực tế, hiệu quả phụ thuộc vào cách triển khai. Việc tính toán gián tiếp thông qua $A^T A$ có thể làm trầm trọng hóa vấn đề điều kiện số và gây mất mát thông tin, dẫn đến sai số đáng kể hoặc hội tụ không chính xác.

Phương pháp Lặp (Iterative Methods)

Gauss-Seidel: Trên hệ SPD, phương pháp hội tụ hiệu quả nhờ bán kính phổ nhỏ hơn 1, với sai số giảm dần theo từng bước lặp. Tiêu chuẩn dừng dựa trên chuẩn L_2 cho phép kiểm soát độ chính xác theo ngưỡng dung sai đã thiết lập.

Tuy nhiên, trên hệ ma trận Hilbert, khi bán kính phổ của ma trận lặp tiệm cận 1, tốc độ hội tụ suy giảm đáng kể hoặc gần như đình trệ. Trong trường hợp này, số vòng lặp lớn kết hợp với sai số làm tròn có thể dẫn đến việc tiêu chuẩn dừng bị thỏa mãn sớm, khiến thuật toán trả về nghiệm không chính xác.

Kết quả cho thấy hiệu quả của Gauss-Seidel phụ thuộc mạnh vào đặc tính phổ của ma trận. Với các bài toán kém điều kiện, phương pháp này không đảm bảo hội tụ nhanh và có thể không đạt được độ chính xác mong muốn trong giới hạn số vòng lặp.

3.4. Kết luận

Thông qua thực nghiệm trên hai lớp bài toán đại diện (hệ SPD và ma trận Hilbert), có thể khẳng định rằng độ ổn định của nghiệm không chỉ phụ thuộc vào bản thân thuật toán mà bị chi phối mạnh bởi số điều kiện $\kappa(A)$ của ma trận.

- **Hệ SPD (well-conditioned)**: Với số điều kiện nhỏ, sai số làm tròn được kiểm soát hiệu quả, cho phép các phương pháp phát huy tối đa đặc tính của mình. Khử Gauss đạt độ chính xác gần giới hạn máy, SVD đảm bảo ổn định số, trong khi Gauss-Seidel hội tụ nhanh đến ngưỡng dung sai đặt trước.
- **Hệ Hilbert (ill-conditioned)**: Số điều kiện lớn dẫn đến khuếch đại sai số nghiêm trọng, làm suy giảm hiệu năng của tất cả các phương pháp. Khử Gauss trở nên mất ổn định, SVD phụ thuộc mạnh vào cách triển khai, và Gauss-Seidel gặp khó khăn trong hội tụ hoặc cho nghiệm kém chính xác.

Kết quả cho thấy không tồn tại phương pháp giải tối ưu cho mọi bài toán. Việc lựa chọn thuật toán cần dựa trên sự cân nhắc giữa độ ổn định số, chi phí tính toán và đặc tính điều kiện của ma trận. Trong đó, số điều kiện $\kappa(A)$ đóng vai trò quyết định trong việc đánh giá độ tin cậy của nghiệm thu được.

CHƯƠNG 4

Kết Luận

4.1. Tóm tắt kết quả đạt được

Trong khuôn khổ đồ án này, nhóm đã nghiên cứu và triển khai các phương pháp cốt lõi trong đại số tuyến tính, tập trung vào hai hướng chính: **giải hệ phương trình tuyến tính bằng phép khử Gauss** và **phân rã ma trận sử dụng phương pháp phân rã giá trị suy biến SVD**.

Ở phần 1, nhóm đã xây dựng thành công các thuật toán dựa trên phép khử Gauss, bao gồm giải hệ phương trình, tính định thức, tìm ma trận nghịch đảo, xác định hạng và không gian con. Việc áp dụng kỹ thuật **partial pivoting** giúp cải thiện độ ổn định số học. Kết quả thực nghiệm cho thấy các thuật toán được cài đặt có sai số rất nhỏ (xấp xỉ 10^{-6}) khi so sánh với thư viện Numpy.

Ở phần 2, nhóm đã tiếp cận một trong những thuật toán phân rã ma trận phổ biến nhất - SVD, đồng thời nhóm cũng đã tìm hiểu thêm một số thuật toán tìm trị riêng như Jacobi và QR. Không chỉ dừng lại ở việc trình bày lý thuyết, nhóm đã tìm hiểu sâu về bản chất hình học của các phép biến đổi tuyến tính. Việc sử dụng Manim để trực quan hóa đã giúp truyền tải các khái niệm trừu tượng thành các hình ảnh sinh động, góp phần nâng cao khả năng tiếp cận và hiểu rõ bản chất phép phân rã.

Ở phần 3, nhóm đã tiến hành phân tích hiệu năng và độ ổn định của các thuật toán đã xây dựng thông qua thực nghiệm. Kết quả cho thấy thời gian thực thi của các phương pháp phù hợp với độ phức tạp lý thuyết, đồng thời các thuật toán trên hoạt động hiệu quả hơn trên các ma trận có điều kiện tốt (SPD) với sai số rất nhỏ. Tuy nhiên, khi áp dụng trên các ma trận điều kiện kém như Hilbert, sai số tăng đáng kể, cho thấy ảnh hưởng rõ rệt của độ điều kiện đến tính ổn định số học.

4.2. Khó khăn gặp phải

Một số khó khăn mà nhóm đã gặp phải trong quá trình tiến hành đồ án:

- Việc sử dụng Manim để xây dựng video trực quan hóa đòi hỏi khá nhiều thời gian để học cách sử dụng và đồng bộ giữa toán học và animation.
- Các vấn đề giải quyết chưa rõ ràng do số lượng tài liệu tham khảo còn hạn chế đối với một số nội dung như thuật toán tìm trị riêng Jacobi.
- Việc kiểm soát sai số số học và so sánh kết quả với Numpy không phải lúc nào cũng dễ dàng.

4.3. Bài học rút ra

Thông qua đồ án, nhóm đã hiểu sâu hơn bản chất của các thuật toán đại số tuyến tính thông qua việc tự cài đặt và kiểm chứng với các thư viện chuẩn. Các khái niệm như không gian vector, hạng ma trận hay SVD trở nên trực quan hơn khi được biểu diễn bằng hình học. Việc sử dụng Manim cũng cho thấy vai trò quan trọng của trực quan hóa trong việc tiếp cận các khái niệm trừu tượng. Ngoài ra, nhóm nhận thức rõ tầm quan trọng của độ ổn định số học và các kỹ thuật như pivoting trong việc đảm bảo tính chính xác của thuật toán.

TÀI LIỆU THAM KHẢO

[1] Gilbert Strang lectures on Linear Algebra (MIT) https://www.youtube.com/playlist?list=PL49CF3715CB9EF31D

[2] Numerical Linear Algebra - Trefethen https://www.stat.uchicago.edu/~lekheng/courses/309/books/Trefethen-Bau.pdf

[3] Matrix Computations - Gene H. Golub

[4] Manim Documentation https://docs.manim.community/en/stable/index.html

[5] 3Blue1Brown https://www.youtube.com/channel/UCYO_jab_esuFRV4b17AJtAw

[6] Alyssa Quek SVD Visualization https://alyssaq.github.io/2015/singular-value-decomposition-visualisation/

[7] SVD and Data Compression using Low-rank Approximation - TheCleverMachine https://dustinstansbury.github.io/theclevermachine/svd-data-compression

[8] SVD Image Compression https://github.com/Tornadosky/svd-image-compression

[9] VanderPlas PCA Scatter Plot https://jakevdp.github.io/PythonDataScienceHandbook/05.09-principal-component-analysis.html

[10] ApplyMatrix - Manim Community https://docs.manim.community/en/stable/reference/manim.animation.transform.ApplyMatrix.html

[11] T. B. Thành, "Bài toán trị riêng trong phương pháp phần tử hữu hạn giải cho hệ dầm liên tục," Luận văn Thạc sĩ Kỹ thuật, Học viện Kỹ thuật Quân sự, Hà Nội, 2008.